

From Static Structures to Ensembles

Studying and Harnessing Protein Structure Tokenization

Liu, Feng, Cao & Li · International Digital Economy Academy (IDEA) · 2025

Ahmet Yiğit Doğan | 31.07.2026

Sequence and Structure

A protein has two faces

- Sequence: a 1D string of amino acids (20-letter alphabet)
- Structure: the 3D shape it folds into (continuous coordinates)

Design tasks need both at once

- Sequence is discrete, structure is continuous. How do you put both into one model?

SEQUENCE: discrete



20-letter alphabet, already tokens

STRUCTURE: continuous

(x, y, z) coordinates for every atom

N: 12.4, 5.1, 8.9 C α : 13.1, 4.7, 9.5 ...

Infinite, continuous, NOT tokens yet

What this paper actually asks

1

How do you best feed structure tokens to a language model?

Turns out the way you represent the input sequence is critical.

2

What is the token vocabulary really like?

Are tokens unique building blocks, or is there hidden redundancy?

3

Can we exploit that vocabulary beyond static prediction?

Yes, to model protein motion, for free.

Structure tokenization with a VQ-VAE

Encoder

Reads the 3D atoms around a residue, outputs a continuous vector (same length as the vectors in codebook).

Codebook

A fixed dictionary of 4096 reference vectors. Snap the encoded vector to the nearest one, its index = the token.

Decoder

Takes token indices → looks up their vectors → rebuilds the 3D structure.

KEY POINT

The dictionary is learned **purely from geometry**. Each token means “a local shape motif” (e.g. ‘a residue in a gently curving helix’), completely **independent of which amino acid sits there**.

The full pipeline: tokenizer + language model

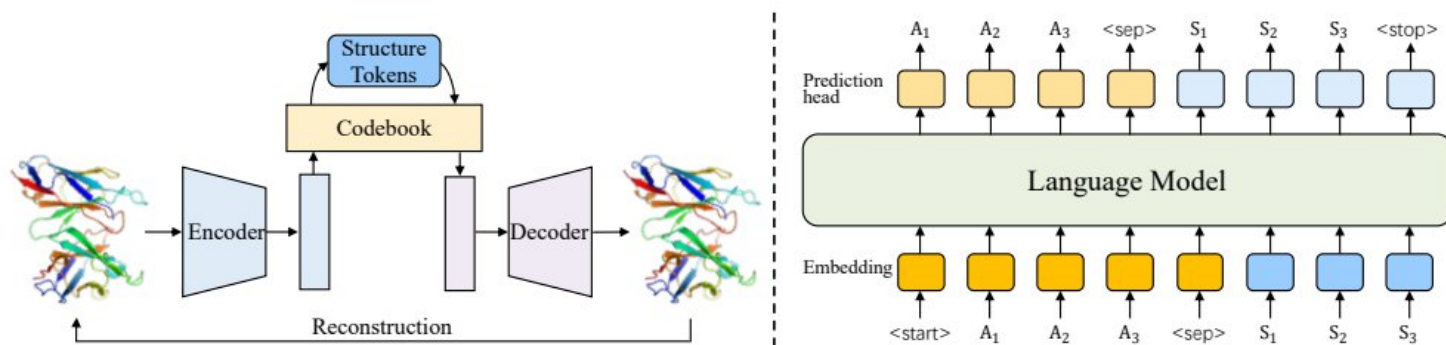


Figure 1: The VQ-VAE (left) discretizes continuous protein structures into a finite set of “Structure Tokens”. These tokens are then used in an autoregressive language model (right) that predicts a sequence of structural tokens conditioned on the amino acid sequence.

Left: the tokenizer

- 3D structure is squeezed to tokens, then rebuilt

Right: the predictor

- A GPT-style model reads amino acids $A_1..A_3$
- then generates structure tokens $S_1..S_3$

A tiny 5-residue peptide

Sequence (the input we always know):



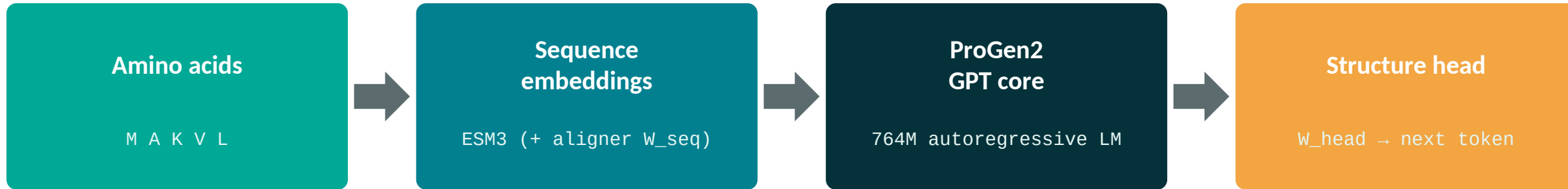
Encode the known 3D structure → each residue snaps to its nearest codebook entry:



M → 1053 A → 2891 K → 77 V → 3410 L → 512

*The structure of our peptide is now the token string [1053, 2891, 77, 3410, 512]. Feed it to the decoder and **reconstruct** the 3D shape.*

A GPT that translates sequence → structure tokens



- The GPT core is ProGen2-medium (764M), a protein language model
- Input structure tokens → embedding → projection matrix W_{struct} into the LM's space
- A new output head W_{head} lets the model emit structure tokens
- **Design choice:** use ProGen2's own sequence embeddings or pre-trained ESM3 embeddings (+ aligner)

Bridging the “semantic gap”

Same model, two ways to represent the input sequence:

ProGen2 embeddings

- Trained only to **predict amino acids**
- Embeddings live on the 'sequence' side only
- **Fails to produce coherent 3D structures**
- TM-score 0.523 on CAMEO

ESM3 embeddings

- ESM3 was trained with **structure awareness**
- Embeddings already straddle both worlds
- **Strong, coherent structure prediction**
- TM-score 0.784 on CAMEO, matches full ESM3

Model	TMScore			RMSD (Å)		
	CAMEO	CASP14	CASP15	CAMEO	CASP14	CASP15
StructGPT	0.523	0.329	0.383	11.87	17.19	18.99
StructGPT w. ESM3_emb	0.784	0.580	0.639	5.43	10.24	11.74
ESM3-sm-open	0.781	0.575	0.625	5.74	10.29	14.69

There is a semantic gap between the sequence “language” and the structure “language”. Structurally-aware embeddings of ESM3 bridge it.

Two-stage training & datasets

1 Align the embeddings

- Freeze the ProGen2 weights
- Train only W_{struct} , W_{seq} , W_{head}
- Data: AFDB SwissProt structures

2 Full fine-tuning

- Unfreeze & update all of ProGen2 + the linear layers
- Data: AFDB + single-chain PDB structures
- Sequences cropped to max length 512

Objective: maximize $P(S_{\text{struct}} \mid S_{\text{seq}})$

Autoregressive factorization: predict each structure token from the amino acids + all earlier structure tokens.

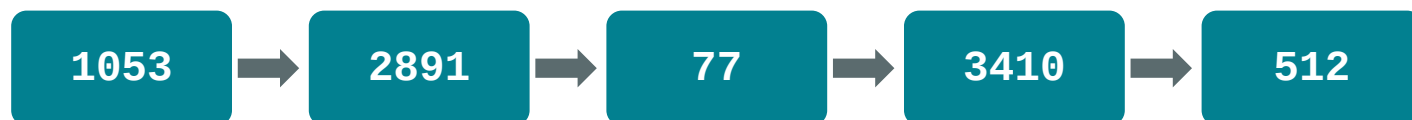
$$P(S_{\text{struct}} \mid S_{\text{seq}}) = \prod_t P(s_t \mid S_{\text{seq}}, s_{<t})$$

What goes in, what comes out

INPUT , just the amino acid sequence

<start> M A K V L <sep>

GENERATE , one structure token at a time (top-p = 0.9, T = 0.7)



predict → append → predict next ... until <stop>

DECODE , tokens → codebook vectors → 3D structure

[FIGURE PLACEHOLDER]

(predicted 3D structure)

The decoder rebuilds the peptide's shape from the generated tokens.

< 5 sec

to predict a 200-residue protein, 100×+ faster than AlphaFold2
(no MSA search)

Structure-prediction performance (Table 1)

Model	TM CAMEO	TM CASP14	TM CASP15	RMSD CAMEO
StructGPT (ProGen2 emb)	0.523	0.329	0.383	11.87
StructGPT w. ESM3 emb	0.784	0.580	0.639	5.43
ESM3-sm-open (reference)	0.781	0.575	0.625	5.74

- ESM3 embeddings roughly **DOUBLE** the TM-score on CAMEO
- ...and halve the RMSD (11.87 $\rightarrow$ 5.43 Å)
- Matches the full open ESM3 model despite a simpler pipeline

TM-score: 0–1, higher = better. RMSD (Å): lower = better.

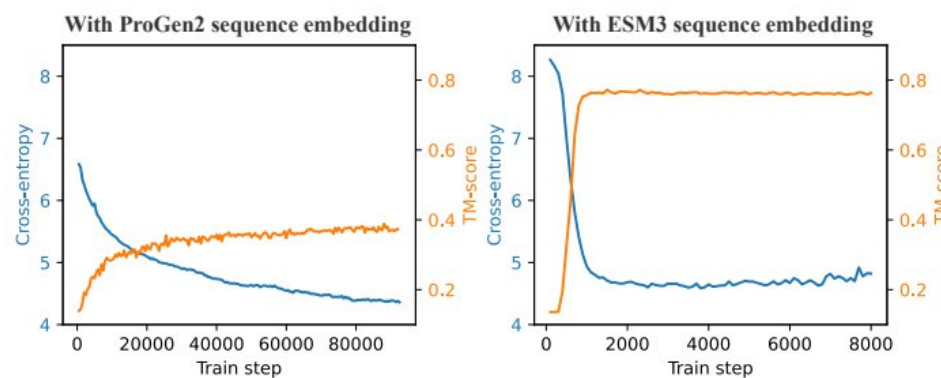


Figure 2: The training curve of the GPT model for protein structure prediction with different sequence embeddings.

Structural “synonyms”

- **The puzzle from Part 1**
 - Cross-entropy loss kept dropping, but structure quality didn't follow
- **The hypothesis**
 - The 4096-entry codebook is REDUNDANT, many different tokens decode to nearly the same local shape
- **Why that hurts training**
 - If the reference uses token 1053 but the model predicts 2210, an equally valid ‘synonym’, the loss still punishes it

Structural synonyms

1053

≈

1054

2210

Different indices, nearly identical geometry.

Their codebook VECTORS sit very close together → they're interchangeable shape-words.

Analogy: “big” vs “large”, different words, same meaning.

The codebook really does cluster (Figure 3)

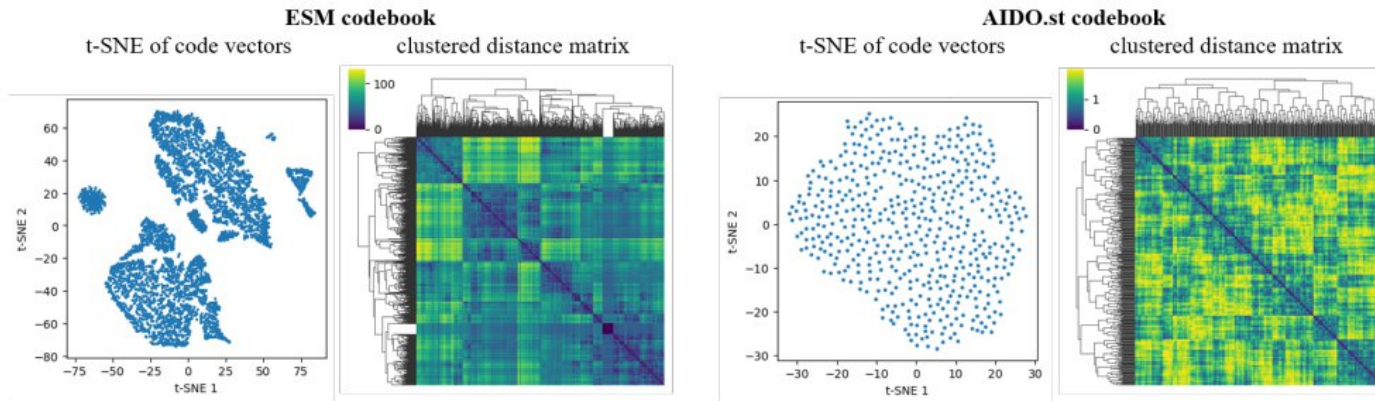


Figure 3: t-SNE visualization and the distance matrix of the code vectors. The ESM3 codebook shows distinct, well-defined clusters, while the t-SNE of the AIDO.st codebook [41] vectors are uniformly distributed in the 2D space.

- **Method 1: t-SNE map**
 - Project the 4096 vectors to 2D → many dense clusters appear
 - Groups of near-identical shape-words
- **Method 2: pairwise distances**
 - Compute distance between every pair of code vectors
 - ESM3 shows strong, well-separated clusters
 - (A rival 512-entry codebook is more diffuse)

The synonym dictionary

- **Definition**

- Two tokens are synonyms if their code vectors are closer than a threshold τ (they use $\tau = 10$)
- For each token, collect its synonym set S_k

- **The proof (perturbation test)**

- Encode a real structure $\rightarrow$ replace each token with a random synonym $\rightarrow$ decode

$$S_k = \{ i : \| v_i - v_k \| < \tau \}$$

Result: swapped structures are almost identical, RMSD often $< 2.0 \text{ \AA}$. Structural equivalence confirmed.

Table 2, structures decoded from synonym-swapped tokens

Dataset	TM-score	RMSD ($\text{\AA}$)
CAMEO	0.933	1.744
CASP14	0.938	1.969
CASP15	0.849	4.136

High TM-scores (0.85–0.94) + low RMSD $\rightarrow$ swapping synonyms barely changes the shape. The redundancy is genuine.

What we're competing with: MD simulations

- **Proteins constantly jiggle & flex, motion is often the function**
 - A single structure is just one snapshot, we want the whole movie
- **MD = Molecular Dynamics**
 - Apply $F = ma$ to every atom, step by tiny step
 - Force field gives forces → update velocities → update positions → repeat
- **Why it's brutally expensive**
 - Time step $\approx$ 1-2 femtoseconds, useful motion needs microseconds
 - $\approx$ 1 billion steps per microsecond $\times$ 100k+ atoms → days-weeks on GPUs

The gold standard...

“A computational microscope”

...but painfully slow.

This paper's goal: get MD-like flexibility info in SECONDS, with no physics and no training.

“Synonym swap”, in four steps

- 1 Encode** Real structure $\rightarrow$ token string `[1053, 2891, 77, 3410, 512]`
- 2 Swap** Replace each token with a RANDOM synonym $\rightarrow$ `[2210, 900, 77, 3411, 512]`
- 3 Decode** Perturbed tokens $\rightarrow$ a slightly different 3D structure
- 4 Repeat** Do it ~ 250 times $\rightarrow$ an ensemble of plausible conformations

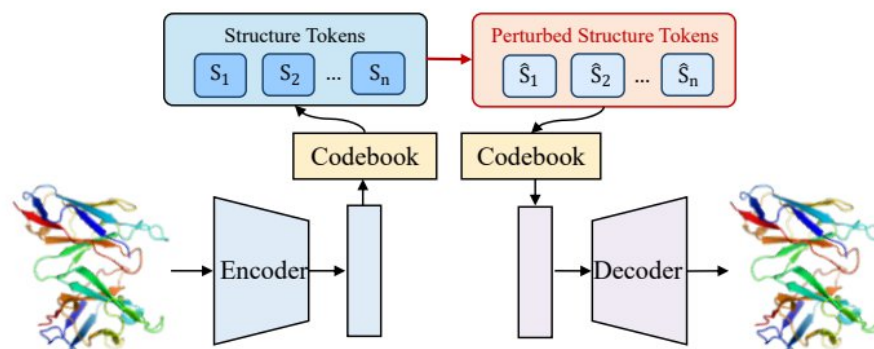


Figure 4: Perturbation of the structure tokens for exploring the conformational ensemble space.

How we judge the ensembles

Setup

- 82 test proteins from the ATLAS database
- Each has ground-truth MD trajectories
- Generate 250 perturbed structures per target

Baselines (both TRAINED on MD data)

- AlphaFlow, AlphaFold + flow matching → samples many folds
- MDGen, generates MD-like trajectories directly
- Our method uses NEITHER training nor physics

Metrics

RMSF

Per-residue flexibility. Correlate with MD (Pearson r). Higher = better.

PCA W_2 dist.

Do we capture the dominant modes of motion?
Lower = better.

Pairwise RMSD

Structure of diversity across the ensemble. Higher r = better.

Competitive, with zero training (Table 3)

Metric	Ours	MDGen	AlphaFlow
Pairwise RMSD $r \uparrow$	0.38	0.48	0.48
Per-target RMSF $r \uparrow$	0.84	0.71	0.85
Global RMSF $r \uparrow$	0.41	0.50	0.50
MD PCA $W_2 \downarrow$	1.83	1.89	1.52
Joint PCA $W_2 \downarrow$	2.54	—	2.25

$\uparrow$ higher better $\downarrow$ lower better · green = our best results

- **Wins where it counts**
 - Per-target RMSF $r = 0.84$, ties AlphaFlow (0.85), beats MDGen (0.71)
 - Best MD PCA W_2 (1.83), captures dominant motions
- **Honest weaknesses**
 - Lower on Global RMSF & Pairwise RMSD, absolute scale of motion is harder
- **...and it does this with NO training, unlike both baselines**

Seeing it work (Figure 5)

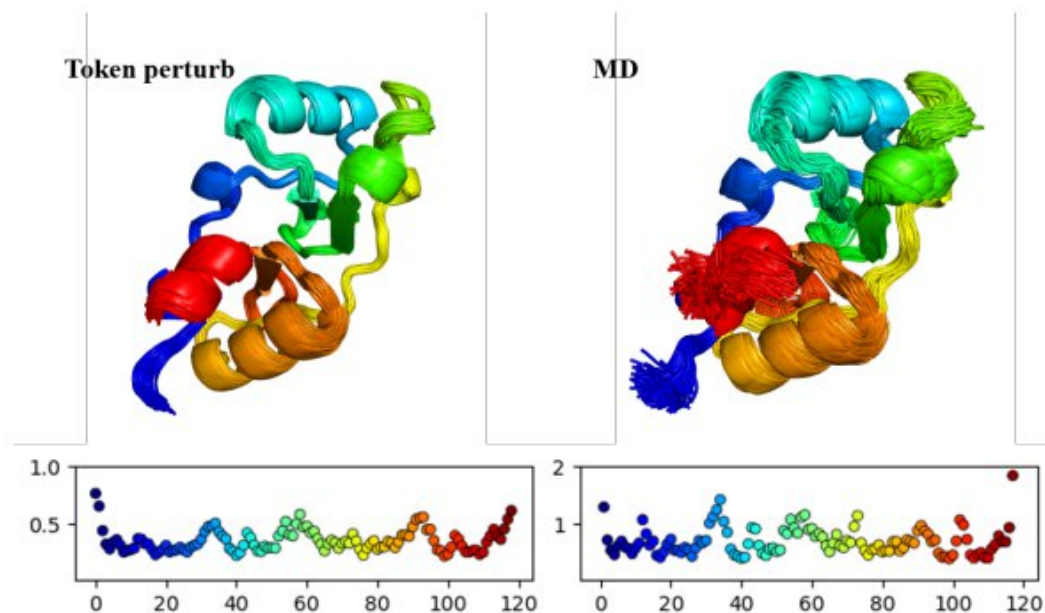


Figure 5: Protein ensembles for 6uof_A generated by token perturbation and MD, and the C α RMSFs indexed by the residue id (Pearson $r = 0.81$).

- **One example protein (6uof_A)**
 - Left: our token-perturbation ensemble
 - Right: the real MD ensemble
- **They look strikingly similar**
 - The RMSF curves track each other closely ($r = 0.81$)
 - The flexible loops and rigid core line up

What the method can't (yet) do



It's a heuristic, not physics

No energy function, no forces, it won't replace MD for mechanistic detail.



Misses cooperative motion

Tokens are perturbed independently, so large-scale, coordinated domain motions are lost.



Absolute scale is shaky

Great at the SHAPE of flexibility (which residues), weaker on the MAGNITUDE.

Future work: perturb token SUB-SEQUENCES (not single tokens), or combine with MSA subsampling to reach large cooperative motions.

Significance & takeaways

1 Scientific insight

Structure tokens aren't a minimal, orthogonal set, they're a redundant 'language' with heavy synonymy. This explains why token-loss misleads and why rich embeddings are needed.

2 A free dynamics tool

Training-free, MSA-free, near-instantaneous ensembles that rival trained models at per-protein flexibility.

3 A reframe

The codebook's redundancy, a 'flaw' for prediction, becomes a feature: a built-in map of protein flexibility.